

Journal Pre-proof

VDMAF: Cross-language source code vulnerability detection using multi-head attention fusion

Yang Li, Qin Luo, Peng Wu, Hongdi Zheng

PII: S0950-5849(25)00078-3

DOI: <https://doi.org/10.1016/j.infsof.2025.107739>

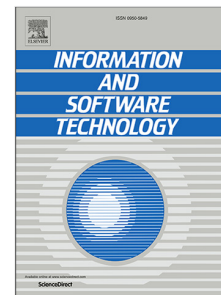
Reference: INFOSOF 107739

To appear in: *Information and Software Technology*

Received date: 25 November 2024

Revised date: 10 March 2025

Accepted date: 1 April 2025



Please cite this article as: Y. Li, Q. Luo, P. Wu et al., VDMAF: Cross-language source code vulnerability detection using multi-head attention fusion, *Information and Software Technology* (2025), doi: <https://doi.org/10.1016/j.infsof.2025.107739>.

This is a PDF file of an article that has undergone enhancements after acceptance, such as the addition of a cover page and metadata, and formatting for readability, but it is not yet the definitive version of record. This version will undergo additional copyediting, typesetting and review before it is published in its final form, but we are providing this version to give early visibility of the article. Please note that, during the production process, errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

© 2025 Published by Elsevier B.V.

VDMFAF: cross-language source code vulnerability detection using multi-head attention fusion

Yang Li^a, Qin Luo^{a,*}, Peng Wu^{b,c,*} and Hongdi Zheng^d

^aSchool of Computer science and software engineering, Southwest Petroleum University, Chengdu, 610500, China

^bKey Laboratory of Sichuan Cuisine Artificial Intelligence, Chengdu, 610100, China

^cSchool of Information and Engineering, Sichuan Tourism University, Chengdu, 610100, China

^dCollege of Electronic and Information Engineering, Southwest Minzu University, Chengdu, 610041, China

ARTICLE INFO

Keywords:

Vulnerability detection
Sequence embedding
Graph neural network
Feature fusion
Multi-Head attention mechanism
Source code representation

ABSTRACT

Context: Detecting potential vulnerabilities is critical for ensuring the stability and reliability of software systems. Traditional static detection methods fall short in accuracy and efficiency. Furthermore, existing deep learning-based vulnerability detection models typically rely on single sequence or graph embedding methods, neglecting the semantic and structured information present in the code. With the diversification of software development environments, systems often involve multiple programming languages. This limits the effectiveness of existing vulnerability detection methods when handling cross-language code.

Objective: To solve these problems, we propose a more effective and general vulnerability detection framework, VDMFAF(Cross-Language Source Code Vulnerability Detection Using Multi-Head Attention Fusion).

Methods: The method extracts unified and standardized feature representations. It uses a multi-head attention module to fuse sequence features and graph structural features. First, an improved global consistent labeling mechanism is introduced, which improves data representation through threshold-based label augmentation. Second, the method uses sequence embedding to extract local semantic features of the code. The code is converted into a unified, standardized graph structure. Then, a graph neural network is used to extract features. Finally, the sequence and graph features are fused using the multi-head attention module, followed by classification with a bidirectional LSTM-based recurrent neural network.

Results: VDMFAF has been evaluated on three vulnerability datasets across different programming languages and granularities, demonstrating better performance across all metrics compared to baseline models, with F1 scores of 98.9%, 65.3%, and 56.8%.

Conclusion: The proposed VDMFAF outperforms state-of-the-art models, exhibiting better generality and scalability, thus showing greater potential in vulnerability detection tasks.

1. Introduction

With the extensive use of external libraries and modules in software development, the complexity of the code is increasing, and vulnerability detection is becoming more and more difficult. Security vulnerabilities in modern software systems can result in significant security risks, as attackers exploit these weaknesses for malicious purposes, including unauthorized system access and data breaches. In 2024, the CVE-2023-46805 and CVE-2024-21887 vulnerabilities were also disclosed. These vulnerabilities mainly affect Ivanti's Connect Secure VPN and Policy Secure devices. Attackers can use them to execute any command, steal data, and establish persistent access with backdoors. Currently, over 162,000 ICS VPN devices are exposed online and face high security risks [1]. In addition, the healthcare sector is also frequently targeted by attacks. Recently, HealthEC, a leading U.S. healthcare technology company, disclosed a large-scale data breach that occurred in July 2023, affecting over 4.4 million people and compromising sensitive information [2]. Consequently, it is critical to find and fix vulnerabilities in your code in a timely manner.

Traditional static detection methods have limitations. They rely on predefined rules to detect vulnerabilities, heavily depending on security experts to manually write these rules. This process is time-consuming and prone to errors [3]. Additionally, the high false positive rate of these tools limits their effectiveness and applicability in real-world production environments [4, 5]. In 2024, a data breach occurred at Finastra. Around 400GB of sensitive financial data was leaked and spread to hacker forums. The incident revealed the limitations of traditional detection tools in handling complex security threats. It highlighted the need for smarter and more efficient vulnerability detection methods [6].

In recent years, the rapid advancement of deep learning (DL) technology has significantly improved vulnerability detection in code [7]. AI-driven threat detection has many advantages, including predictive threat detection, improved accuracy and scalability, reduced response time, and the ability to adapt to evolving threats [8]. For example, VulDeePecker [9] and SySeVR [10] have demonstrated exceptional performance in vulnerability detection by using neural networks for automatic feature extraction. However, these methods treat source code as a flat sequence of natural language, capturing only sequence features and neglecting the inherent structural information within the code. This limitation brings challenges in addressing complex code

*Corresponding author

✉ doroathy_1q@163.com (Q. Luo); scetc_2012@163.com (P. Wu)
ORCID(s):

structures and diverse vulnerability patterns. Subsequently, Devign [11] and Reveal [12] utilized graph neural networks to capture information from graph structures. Nevertheless, graph-embedding models often overlook the semantic information embedded in node statements. Traditional Graph Neural Networks (GNNs) aggregate information from neighboring nodes, limiting the propagation of structural information and failing to capture complex vulnerability patterns in code. The Vul-LMGNNs [13] method proposes combining language models (codeLMs) with GNNs, where the language model provides semantic information and the GNN handles structural information, enabling more comprehensive vulnerability detection. Recently, some vulnerability detection methods, such as CSGVD [14] and MFXSS [15], have started combining both sequential and graph structural features. CSGVD uses graph neural networks to extract structured information and introduces Average Bidirectional Affine Attention Pooling (M-BFA) to enhance node information aggregation as a feature representation of the graph. MFXSS combines Abstract Syntax Trees (AST) and Control Flow Graphs (CFG) to convert generalized sample data into graph structures and code string structures. It extracts and fuses logical calling features and contextual execution relationship features using graph convolutional networks, weighted aggregation, and bidirectional recurrent neural networks. Although both methods effectively combine features from sequential and graph structures, they rely on simple fusion techniques, such as averaging or direct summation, for multi-dimensional feature fusion, lacking an effective approach for combining two-dimensional features. Some methods also use ensemble learning to fuse information from multiple dimensions [16]. However, their main drawback is high computational complexity. Especially when handling large-scale data, training multiple models requires significant computational resources, which may affect the model's practical application efficiency. Most existing methods are developed for C/C++ languages. Some work has explored vulnerability detection in other programming languages [17, 18], but few methods address cross-language detection. Initially, FUNDED aimed to collect high-quality training samples automatically from open-source projects by combining probabilistic learning and statistical evaluation [19]. It used multi-relational graph representation learning to improve detection accuracy and cross-language generalization. However, its model requires transfer learning. IRC-CLVul moves away from transfer learning and proposes a cross-language detection method based on intermediate representations [20]. It eliminates vocabulary differences by converting programs from different languages to a unified LLVM intermediate representation (LLVM-IR). However, its model requires a complex compilation process to handle the source dataset. To optimize feature fusion and improve cross-language detection, more advanced technologies and tools are needed to further enhance detection accuracy and generalization.

This paper proposes a simple and effective cross-language vulnerability detection framework, VDMAF, aimed at effectively combining the semantic and structural information of code to provide a more general and more accurate vulnerability detection method. This method uses a multi-head attention to combine the features of sequence embeddings trained by Word2Vec model with graph embeddings trained by gated graph neural network (GGNN). The fused features are then input into a bidirectional LSTM recurrent neural network with an attention mechanism for classification. The representations of sequences and graphs work together to improve feature extraction. This enhances the ability to capture complex patterns and structures in the code. As a result, the accuracy of vulnerability detection is improved, and false positive rates are reduced. Experiments demonstrate that VDMAF outperforms baseline model methods across different programming languages and vulnerability datasets with varying granularities.

In summary, the main contributions of this paper are as follows:

- We propose a general and flexible cross-language vulnerability detection framework, VDMAF. The framework aims to achieve cross-language vulnerability detection by extracting unified and standardized feature representation and introducing a multi-head attention mechanism with better generalization capability to focus on different feature subspaces.
- We construct a globally consistent labeling strategy to improve the accuracy of the data slicing and labeling process in source code vulnerability detection, enabling the model to aggregate information more effectively.
- We propose a multi-head attention feature fusion module that captures multi-dimensional and multi-scale feature relationships through different attention heads, thereby improving the model's expressive ability.

2. Related work

2.1. Traditional methods

Early techniques for security vulnerability detection can be classified into two primary categories: static analysis and dynamic analysis. Static analysis identifies potential security vulnerabilities by examining source code or binary files without executing the program. This approach typically employs methods such as symbolic analysis, taint analysis, formal verification, and rule-based checks. The advantages of static analysis include its ability to detect vulnerabilities early in the development process, comprehensive coverage of all code paths, and operation without requiring program execution. However, it is also associated with a high rate of false positives and challenges in identifying unknown vulnerabilities. Dynamic analysis detects vulnerabilities by monitoring a program's behavior during execution, utilizing techniques such as runtime monitoring, fuzz testing,

behavioral analysis, and simulated attacks. The strengths of dynamic analysis lie in its ability to identify runtime vulnerabilities that depend on specific inputs or environments, leading to a lower false positive rate. The main drawback is that it requires the actual execution of the program, relying on the quality of test cases. This leads to high costs and makes it difficult to cover all execution paths of the program.

2.2. Deep learning based approach

Detection methods based on traditional machine learning face challenges. They rely on domain experts to manually design and extract features. Additionally, these methods are often limited to known vulnerability patterns, making it difficult to effectively address new and complex attack techniques. In the realm of code vulnerability detection utilizing deep learning techniques, most researchers extract key features from the sequential structure of code by transforming source code into a linear sequence and applying natural language processing (NLP) methods for feature extraction. VUDENC [17] uses the Word2Vec model to identify semantically similar code tokens and uses LSTM networks for fine-grained classification of vulnerable code token sequences. This approach significantly enhances the accuracy and efficiency of vulnerability detection by automatically learning features from a large corpus of real Python code. VulDeePecker [9] uses a deep learning-based methodology that converts code snippets into vector representations, thereby simplifying the complexity of manually defining features and achieving a low false negative rate with a reasonable false positive rate. This method emphasizes enhancing the comprehensiveness and practicality of vulnerability detection through automation. μ VulDeePecker [21] introduces a code attention mechanism, facilitating fine-grained detection of multiple vulnerability types via deep learning. This approach not only identifies vulnerabilities within programs but also accurately classifies their types, thereby improving the capability and precision of vulnerability detection. SySeVR [10] systematically detects vulnerabilities in C/C++ programs by combining syntactic and semantic information with deep learning techniques. This method particularly aims to enhance both the efficiency and comprehensiveness of vulnerability detection, including the identification of vulnerabilities not documented in national vulnerability databases.

The study found that the hybrid neural network model extracted more feature information, leading to improved detection performance. These methods effectively capture the contextual information associated with vulnerabilities in the source code. However, they often neglect the structural features of the source code, leading to substantial loss of semantic information during code representation and feature extraction.

2.3. Graph neural network based approach

To address the neglect of structural features in vulnerability detection, researchers have increasingly leveraged graph neural networks (GNNs) to extract richer semantic information, thereby identifying key features from the

complex structures of code. Techniques for vulnerability detection based on GNNs can be categorized into three primary steps: converting source code into a graph format, vectorizing the graph, and training deep learning models using GNNs. Methods such as Devign [11], Reveal [12], BGNN4VD [22], and AI4VA [23] utilize GNNs to extract feature vectors from code. These approaches effectively capture complex dependency relationships within code structures by constructing abstract syntax trees (ASTs) and program dependency graphs (PDGs). By integrating ASTs and PDGs, these methods comprehensively mine contextual information from the code, thereby enhancing the accuracy and efficiency of models in code classification tasks. Additionally, these models exploit the properties of GNNs to automatically learn patterns and features within code snippets, allowing them to effectively manage the intricate structures and relationships found in large-scale codebases, thus improving vulnerability detection performance. Furthermore, CSGVD [14], VulSPG [5], and ACGVD [24] use multiple attention mechanisms to enhance the comprehensiveness and accuracy of vulnerability detection through more refined feature extraction. These methods aim to aggregate node information more effectively, capturing essential patterns and details in the code structure, thereby improving model performance in complex coding environments. The application of various attention mechanisms enables these approaches to address multi-level dependency relationships within code, enhancing both the accuracy of code representation and the adaptability of the models across different contexts and scenarios. Poem [25], GRACE [26], and IvDetect [27] further bolster the generalization capabilities of vulnerability detection models and their performance in complex coding scenarios by integrating multi-level structural information. To mitigate challenges arising from variations in compilers and optimization levels, VulHawk [28] and HCRVD [29] focus on cross-architecture and cross-language vulnerability detection, thereby enhancing model adaptability and universality.

Despite the notable performance of these methods in code vulnerability detection, several challenges remain in practical applications, including complex code structures, scarcity of cross-project data, the demand for real-time detection, noise handling, and data privacy. In response, researchers have proposed a range of innovative solutions in recent years to address these issues and further enhance the performance and adaptability of vulnerability detection, including FUNDED [19], CodeJIT [30], CSVD-TF [31], AMPLE [32], SeDPKG [33], MSGVul [34], and VDBFL [35]. Collectively, these studies represent significant advancements in the extraction of graph-structured vulnerability features within the domain of static code analysis.

3. VDMAF Framework

In this section, we begin by defining the problem. Next, we describe the overall framework of the VDMAF. Lastly, we explain the different components of the model.

3.1. Problem definition

We define code vulnerability detection as a binary classification problem that aims to classify blocks of source code as vulnerable and non-vulnerable. In the code vulnerability detection problem, the dataset D consists of a number of pairs (C_i, y_i) , where C_i is the code function; y_i denotes the vulnerability classification label, $y_i \in \{0, 1\}$, 0 means no vulnerability, 1 means vulnerability. The goal of code vulnerability detection is to train a neural network $N_f(C)$, continuously reduce the vulnerability classification loss and eventually achieve accurate classification of vulnerabilities in source code. The above loss can be formally defined as follow.

$$\min_{N_f} \frac{1}{N} \sum_{(C_i, y_i) \in D} L(N_f(C_i), y_i) \quad (1)$$

where $L(\cdot)$ is the code vulnerability classification loss, y_i is the vulnerability classification label, $N_f(C_i)$ denotes the predicted output of the neural network for the code function C_i , N is the number of samples in the data set.

The goal of this loss function is to minimize the vulnerability classification loss on dataset D by optimizing the parameters of the neural network to improve the performance of the model in the source code vulnerability detection task.

3.2. Framework proposed

The proposed framework comprises five phases: data source, sequential feature extraction, graph feature extraction, multi-head attention feature fusion, and deep learning classification model. In the data source phase, the code is initially sliced based on context, followed by labeling with predefined thresholds. The sequence feature extraction Phase uses the word embedding model Word2Vec to extract features from the code sequence and generate the sequence feature representation. In the graph feature extraction phase, we use gated graph neural network (GGNN) for graph embedding of the code property graph, extract the deep features of the code property graph based on the node feature vectors, and output the feature vector representation of the graph. The multi-head attention feature fusion phase combines the sequence and graph features using a multi-head attention, effectively capturing the interrelationships among the various features. In the deep learning classification model phase, the fused features previously extracted are used to train a recurrent neural network with Bi-LSTM. Ultimately, it is used for the code vulnerability detection task. The VDMAF framework is shown in Fig. 1, and the architecture of the model part is shown in Fig. 2. In the following sections, we will provide a detailed description of each component of the model.

3.2.1. Data source

In the data source phase, we preprocess various datasets. The Bagheri dataset comprises commits that contain security fixes. In each commit, changed or deleted code is designated as "vulnerable," while the remaining code and the revised version are labeled as "(possibly) non-vulnerable."

To ensure accurate data processing and prevent dataset imbalance, the labeling process must treat vulnerable and non-vulnerable sections in a consistent manner and proportion. The initial strategy involves segmenting the data. If a segment overlaps with a vulnerable code section, it is marked as vulnerable; otherwise, it is marked as clean.

The labeling process is shown in Fig. 3. A sliding window of length n traverses the entire source code. The five positions of the focus window are indicated in blue. The focus window begins and ends at code delimiter characters (such as colons, parentheses, or spaces) to avoid cutting through delimiters. The context length of the window is m , which starts and stops at code delimiter boundaries, where $m > n$. If the focus window is near the beginning of the file, the context primarily lies behind it (e.g., Block A); if it is located in the middle, the context is evenly distributed around the code, resulting in some overlapping blocks. If an entire block contains vulnerable code, it is marked as vulnerable; otherwise, it is labeled as clean, ensuring accurate labeling of code segments that contain vulnerabilities. Generally, a block of 10 lines of code is adequate to capture the relevant context for vulnerabilities [36].

To improve labeling accuracy and better aggregate node information, we propose a globally consistent revised labeling strategy that introduces a threshold I_v . Building upon the original labeling method, if an entire block contains vulnerable code, it is labeled as "vulnerable." Furthermore, if the distance between the vulnerable code segment and the block does not exceed the threshold I_v , the block is also designated as "vulnerable". This improvement broadens the contextual range of vulnerable code, ensuring that code fragments containing vulnerabilities are accurately labeled while also capturing adjacent code blocks related to the vulnerable code. This improvement guarantees that the labeling strategy maintains global consistency, thereby improving the accuracy and overall performance of vulnerability detection. Through experimental optimization of the parameter configurations for n , m , and I_v , we identified the most effective labeling strategy, resulting in improved performance of the model in identifying and detecting potential vulnerabilities in the code. The pseudocode for this labeling algorithm is shown in Algorithm 1.

3.2.2. Sequence feature extraction

Comprehensive and detailed code representation plays a crucial role in vulnerability detection. Sequence-based methods can fully leverage the semantic information contained in the source code. For sequence feature extraction, we choose Word2Vec as the sequence-based node embedding method. This study selects Word2Vec instead of CodeBERT for sequence embedding. The decision comes from their fundamental differences in representation learning and alignment with task objectives. Word2Vec models word vectors through co-occurrence statistics in local context

Short Title of the Article

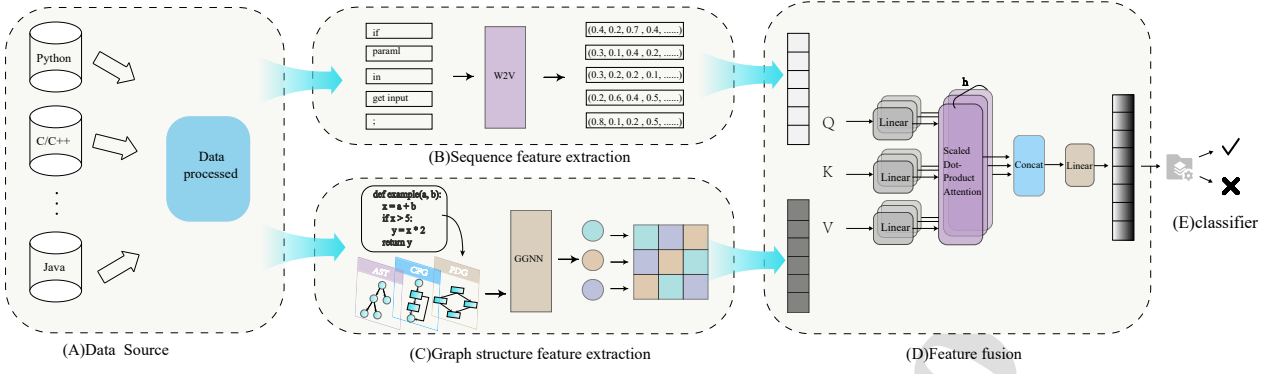


Fig. 1: VDMAF framework.

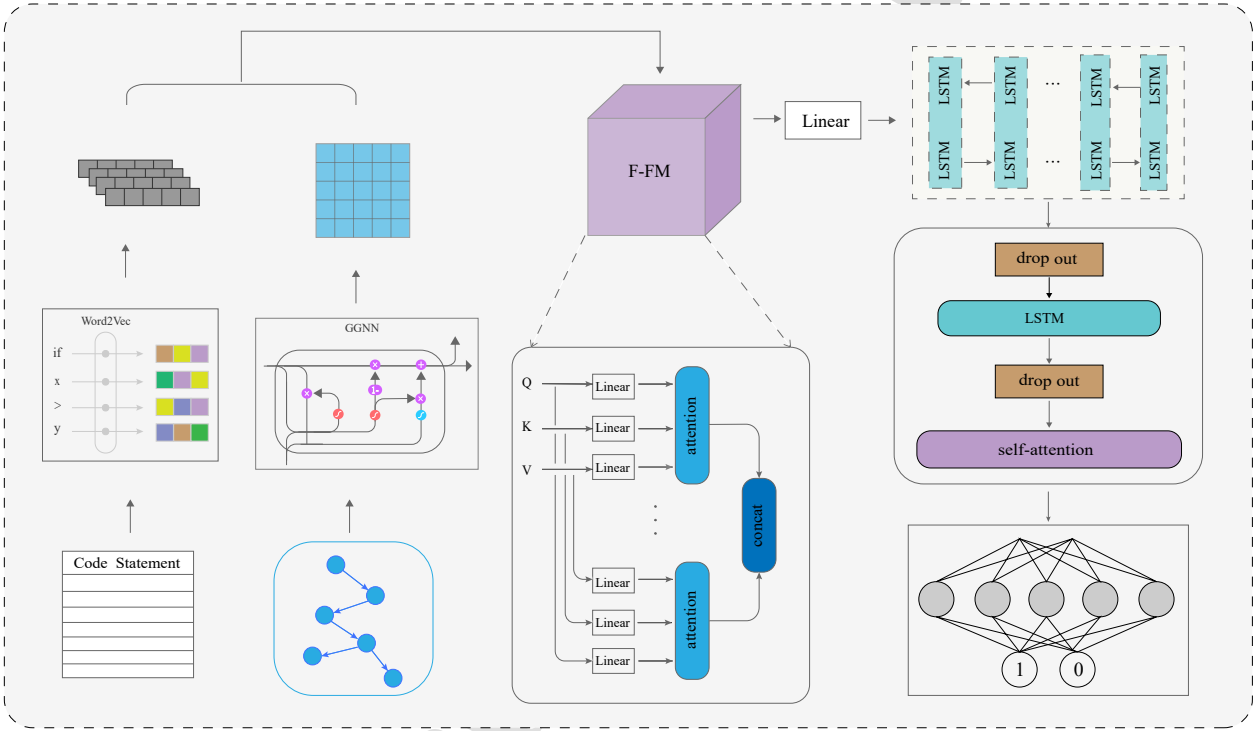


Fig. 2: Model architecture..

windows. Its objective function maximizes conditional probability.

$$\max \sum_{(w,c) \in D} \log P(c|w) = \sum_{(w,c) \in D} \frac{\exp(v_c \cdot v_w)}{\sum_{c' \in V} \exp(v_{c'} \cdot v_w)} \quad (2)$$

Here, v_w represents center word vectors, v_c represents context word vectors, and D denotes training corpus. This shallow neural network generates static embeddings as fixed vector E . Its linear interpretability strongly captures local code patterns. Examples include API call sequences or repeated syntactic structures. In contrast, CodeBERT uses transformer-based dynamic representations with self-attention mechanisms. Although it models long-range dependencies (e.g., cross-function variable propagation), its

highly nonlinear representations may conflict with this study's main goal. Our core objective is verifying cross-language feature fusion effectiveness. We aim to explore interaction patterns between sequence features and graph structural features through multi-head attention. The study does not seek maximum performance from single embedding modules. Choosing more deterministic Word2Vec helps control variables. It avoids interference from pre-trained model parameter randomness in feature fusion. Cross-language generalization forms another key reason for selecting Word2Vec. The embedding method needs language independence and low-resource adaptability. Vulnerability detection essentially maximizes mutual information $I(X; Y)$ between code patterns and vulnerability patterns.

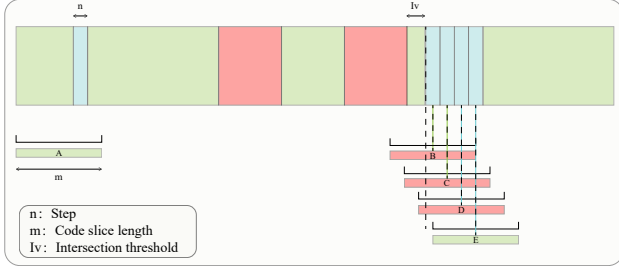


Fig. 3: A globally consistent label generation strategy.

Algorithm 1 Labeling algorithms based on global consistency

Require: Code $\mathbb{C}$, list of vulnerable code lines $\mathcal{V}$, focus window length n , context window length m , labeling threshold I_v

Ensure: Sliced code fragments and their labels S

```

1:  $S \leftarrow []$  {Initialize an empty list  $S$  to store the code
   fragments and their labels}
2: for  $i$  from 0 to  $\text{length}(\mathbb{C})$  step  $n$  do
3:   Fragment  $s \leftarrow \mathbb{C}[i : i + n]$ 
4:    $\text{start\_line} \leftarrow i + 1$ 
5:    $\text{end\_line} \leftarrow i + n$ 
6:    $S.\text{append}((s, \text{start\_line}, \text{end\_line}, 0))$ 
7: end for
8: for each  $(s, \text{start\_line}, \text{end\_line}, \text{label})$  in  $S$  do
9:    $\text{is\_vulnerable} \leftarrow \text{False}$ 
10:  for each  $v$  in  $\mathcal{V}$  do
11:    if  $\text{start\_line} - I_v \leq v \leq \text{end\_line} + I_v$  then
12:       $\text{is\_vulnerable} \leftarrow \text{True}$ 
13:      break
14:    end if
15:  end for
16:  if  $\text{is\_vulnerable}$  then
17:     $\text{label} \leftarrow 1$ 
18:  end if
19: end for
20: return  $S$ 

```

Here, X represents code features and Y denotes vulnerability labels. Word2Vec local window training maximizes $I(X_{\text{local}}; Y)$. CodeBERT cross-language ability depends on its pretraining corpus coverage. It may inadequately represent niche languages (e.g., Solidity for smart contracts). CodeBERT global pretraining optimizes $I(X_{\text{global}}; Y)$. Existing experiments show $I(X_{\text{local}}; Y) > I(X_{\text{global}}; Y)$ in vulnerability detection [37]. This proves local patterns contribute more to vulnerability signals. Our cross-language goal reinforces this choice. language set $\mathcal{L}$, Word2Vec aligns feature spaces by training separate embedding matrices for each language

$$E^{(l)} \in \mathbb{R}^{|V^{(l)}| \times d} \quad (l \in \mathcal{L}) \quad (3)$$

```

1  def simple_example(a, b):
2      x = a + b
3      if x > 5:
4          y = x * 2
5      else:
6          y = x - 2
7      return y

```

Fig. 4: code example.

CodeBERT requires cross-language pretraining condition $\bigcup_l \mathcal{D}_{\text{pretrain}}^{(l)} \supseteq \mathcal{L}$. This causes representation degradation for new languages $l_{\text{new}} \notin \mathcal{D}_{\text{pretrain}}$.

Given a code fragment C , its sequence representation is denoted as $[w_1; w_2; \dots; w_n]$, where W_i represents the i -th token. The Word2vec model is used for map each token W_i into a d -dimensional vector space, resulting in the word vector representation $[v_1; v_2; \dots; v_n]$. The sequence feature representation of the code fragment C is the combination of these word vectors, the formula is as follows:

$$X_{\text{seq}} = [v_1; v_2; \dots; v_n] \quad (4)$$

where ";" denotes the concatenation operation of vectors. The sequence feature representation method can effectively capture the semantic information in the code, which aids downstream vulnerability detection tasks.

3.2.3. Graph feature extraction

In the feature extraction phase of vulnerability detection, we use Graph Neural Networks (GGNN) for the graph embedding of the Code Property Graph (CPG). Our method uses the Joern [38] tool to convert source code into a code property graph. Take the function `simple_example` in Fig. 4 as an example, its Abstract Syntax Tree (AST), Control Flow Graph (CFG), Data Flow Graph (DFG) and Program Dependency Graph (PDG) are shown in Fig. 5. Since the Program Dependency Graph inherently includes the information from the Data Flow Graph, only the AST, CFG, and PDG structures are utilized in the final synthesis of the Code Property Graph. The resulting Code Property Graph for the example code is displayed in Fig. 6. Finally, we leverage GGNN to extract deep features from the Code Property Graph based on node vectors, producing a feature vector representation of the graph.

Given a code fragment C , its CPG is represented as $G = (V, E)$, where V represents the set of nodes and E represents the set of edges. Each node v in V has an initial feature vector h_v^0 . The GGNN updates the feature vectors of each node v in V , with the update equation as follows:

$$h_v^{(t)} = \text{GRU}(h_v^{(t-1)}, \sum_{u \in N(v)} W h_u^{(t-1)}) \quad (5)$$

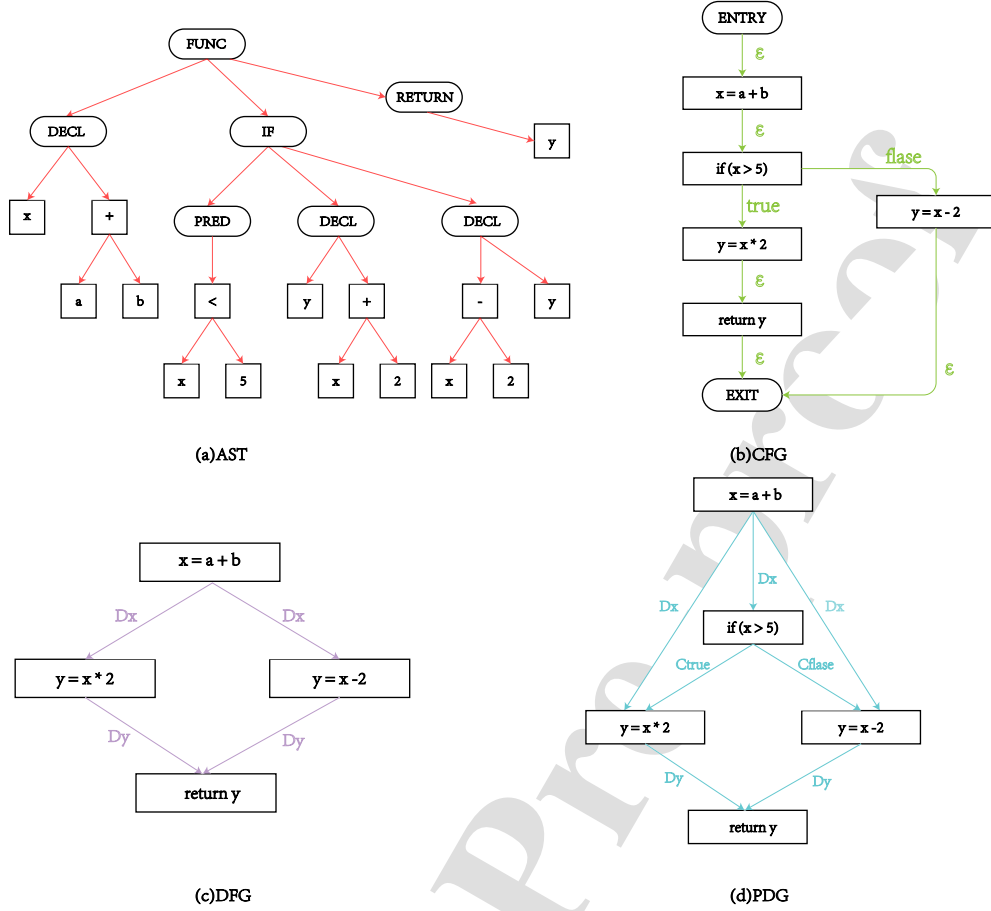


Fig. 5: An example of CPG derived from the code example function in Figure 3. The red solid line represents AST, the green dashed line represents CFG, and the blue dashed line represents PDG.

where GRU represents the Gated Recurrent Unit, $N(v)$ represents the set of neighboring nodes of v , and W is the learnable weight matrix. The update of the GRU is as follows:

$$z^{(t)} = \sigma(W_z \cdot [h_v^{(t-1)}, x_v] + b_z) \quad (6)$$

$$r^{(t)} = \sigma(W_r \cdot [h_v^{(t-1)}, x_v] + b_r) \quad (7)$$

$$\tilde{h}_v^{(t)} = \tanh(W \cdot [r^{(t)} \circ h_v^{(t-1)}, x_v] + b) \quad (8)$$

$$h_v^{(t)} = (1 - z^{(t)}) \circ h_v^{(t-1)} + z^{(t)} \circ \tilde{h}_v^{(t)} \quad (9)$$

$z^{(r)}$ and $z^{(t)}$ are the update and reset gates, and σ is the sigmoid function, $\circ$ denotes element-by-element multiplication. After T iterations, the global feature representation of the graph is as follows:

$$X_{graph} = \sum_{v \in V} h_v^{(T)} \quad (10)$$

We convert the code to CPG, which combines the properties of AST, CFG, DFG and PDG. Then, we use the GGNN to extract feature embeddings. The initial feature vector h_v^0 for each node v in the CPG can be represented by the word embedding of the code fragment or other semantic

information. Through multiple iterations of updating node feature vectors, the GGNN captures both the structural and semantic information of the code fragment. Finally, the feature vectors of all nodes are aggregated into the global feature representation X_{graph} , which is used for downstream vulnerability detection tasks.

3.2.4. Multi-head attention feature fusion

In this method, integrating the sequential features and graph structural features of code is a crucial step. Through the multi-head attention mechanism, we can effectively capture the relationships between different features, ultimately forming a fused feature representation.

Sequence embeddings and graph embeddings represent different aspects of the code. Sequence embeddings focus on lexical and syntactic structures as well as execution order, while graph embeddings describe dependencies and topological structures among code elements. Effectively integrating these two modalities and leveraging their complementary information is a key challenge. Traditional methods, such as simple concatenation or weighted summation, may fail to model their complex interactions. The multi-head attention mechanism provides a flexible and efficient fusion

approach, enabling adaptive learning of the relationships between sequence and graph features.

The sequence embedding X_{seq} is used as the Query, while the graph embedding X_{graph} serves as both the Key and the Value. The multi-head attention mechanism is applied to compute the attention weights between them, The calculation formulas are provided in Eqs.(9)-(10):

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (11)$$

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V) \quad (12)$$

Here, Q , K , and V represent the Query matrix, Key matrix, and Value matrix, respectively. The dimension of the Key vector is denoted as d_k . The trainable weight matrices W_i^Q , W_i^K , and W_i^V correspond to the i -th attention head. This design aims to compute the attention weights of code sequence features on different graph structural features, determining which graph features a specific code token should extract information from. This mechanism allows different tokens to focus on different structural information and dynamically adjust fusion weights based on varying information needs across code positions. The multi-head attention computes adaptive fusion weights according to the relationships between the Query and Key, ensuring a rational combination of different types of information. By concatenating the outputs of multiple heads and applying a linear transformation, the final fused feature representation is obtained, where W_o is the learnable weight matrix for the output.

$$MultiHead(Q, K, V) = \text{Concat}(head_1, head_2, \dots, head_h)W^O \quad (13)$$

The computation of the fused features follows Equation (12), where the sequence features serve as the Query matrix, while the graph structural features act as both the Key and Value matrices.

$$X_{fused} = MultiHead(X_{seq}, X_{graph}, X_{graph}) \quad (14)$$

By computing multiple attention heads in parallel, the model can simultaneously capture both local contextual information and global structural dependencies in the code. This enhances the richness of the fused features. Traditional fusion methods fail to distinguish which graph features are more important for a given code token. In contrast, the attention mechanism adaptively learns weights, allowing sequential features to focus on the most relevant graph information, thereby improving representation capability. Moreover, semantic relationships in code often involve long-range dependencies, such as cross-function calls and cross-module variable usage. The attention mechanism enables direct computation of these global interactions.

3.2.5. Deep learning classification model

In the deep learning classification mode phase, combining the fusion features extracted from multi-head attention

above, VDMAF uses recurrent neural network with Bi-LSTM as the classifier. It also includes a dropout layer and attention mechanism. The loss function is defined as the weighted cross-entropy (wce), and the weighted cross-entropy function is defined as follows:

$$\zeta_{wce} = - \sum_{i=1}^N w_i \cdot y_i \cdot \log(p_i) \quad (15)$$

where w_i represents the class weight for class i , which is designed to improve the detection ability of minority classes. y_i is the true label, if the sample belongs to class 1, $y_i = 1$, otherwise $y_i = 0$, and p_i is the predicted probability for class i .

At this phase, our goal is to train a classification model that can learn from the previously extracted fused feature representation. The fused feature representation X_{fused} is fed into the Bi-LSTM layer to obtain the output H , as shown in Eq(15). Then, the attention mechanism is used to weight the feature representations of the forward and backward contextual features. The attention mechanism assigns different weights to each time step's output of the BiLSTM to emphasize the more important features for the classification task. The attention weights α_i and the contextual feature representation C are defined as follows in Eqs.(14)-(15).

$$\alpha_i = \frac{\exp(\tanh(W_h h_i + b_h))}{\sum_{j=1}^n \exp(\tanh(W_h h_j + b_h))} \quad (16)$$

$$Attention(H) = \sum_{i=1}^n \alpha_i h_i \quad (17)$$

where h_i is the LSTM output at time step i , W_h and b_h are the learnable weight matrix and bias vector.

After the Dropout layer and the fully connected layer, the final classification result is generated. The classification model can be constructed as shown in Eq.(17) to predict whether a given code block contains vulnerable code or not.

$$H = BiLSTM(X_{fused}) \quad (18)$$

$$Y = \sigma(Dropout(W_{fc} \cdot$$

$$Attention(BiLSTM(X_{fused})) + b_{fc})) \quad (19)$$

where W_{fc} and b_{fc} are the learnable parameters of the fully connected layer $\sigma(\cdot)$ is the activation function, and $Y \in \{0, 1\}$ denotes the final prediction result.

Through the above stages, the VDMAF method can effectively extract and fuse multiple features of the source code to finally categorize the code blocks that contain defects and those that do not.

4. Experimental setting

In this section, we present information regarding the datasets and the relevant experimental setup.

4.1. Dataset

For the experiments, we utilized the C/C++ datasets: Devign [11] and Reveal [12], the Python dataset: Bagheri

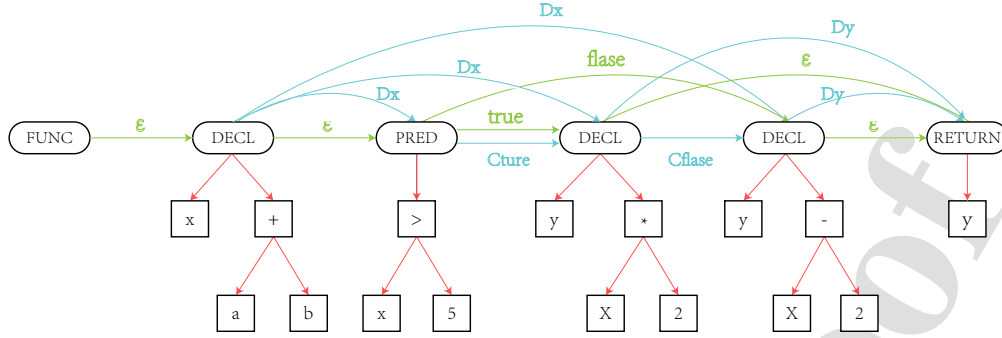


Fig. 6: An example of CPG derived from the code example function in Figure 3. The red solid line represents AST, the green dashed line represents CFG, and the blue dashed line represents PDG.

Table 1

C/C++ Vulnerability Datasets of Functional Granularity: Design and Reveal.

Dataset	Target	Func	Pos.	Neg.	Ratio(%)
Devin	ffmpeg	9769	4788	4981	50.98
	qemu	17549	10070	7479	42.61
Reveal	debian	18298	16883	1415	8.38
	chrome	4436	3611	825	22.9

* Pos. refers to "Positive Class", Neg. refers to "Negative Class."

[17], and the Android(Java) dataset: LVDA [39]. These datasets have been extensively used in vulnerability research, including studies such as GRACE [26] and VU-DENC [17]. Detailed information about these three datasets is provided below.

Devin: The Devin dataset is designed for vulnerability detection in C language, derived from the FFmpeg and QEMU projects. It extracts labeled functions from security-related commits.

Reveal: The Reveal dataset is sourced from the Linux Debian Kernel and Chromium projects, based on vulnerability fix patches associated with these projects.

Bagheri: The Bagheri dataset focuses on Python projects, extracting vulnerability-related commits through the GitHub API and the PyDriller tool. It serves as an evaluation and training platform for Python code vulnerability detection models.

LVDA: The LVDA dataset is generated by scanning over 15,000 Android applications. It contains more than 2 million unique code samples, labeled according to the Common Weakness Enumeration (CWE).

Detailed information about the Reveal and Devin datasets is shown in Table 1. The number of repositories, commits, functions, and the proportion of vulnerabilities contained in the Bagheri dataset are summarized in Table 2. The vulnerability types, number of containers, and lines of code in LVDA are shown in Table 3.

4.2. Experimental setting

The samples were divided into training, validation, and testing sets in a 6:2:2 ratio. The model achieving the highest

F1 score on the validation set was designated as the best model. The mincount is 5, the dimension is 300, the epoch is 100, the dropout is set to 0.2 to prevent overfitting, the learning rate is set to 0.001, and the optimizer is Adam. Training was terminated if the F1 score on the validation set did not show improvement over ten consecutive iterations.

Parameter settings for each phase of the VDMAF framework are presented in Table 4.

4.3. Baselines

In this paper, we compare the performance of VDMAF with six sequence-based models, five graph-based models, respectively.

Sequence-based model:

- **VulDeePecker** [9]: a deep learning-based vulnerability detection method uses a CNN to extract syntactic features of code and a LSTM to capture contextual information.
- **SySeVR** [10]: a graph neural network-based vulnerability detection method constructs a program's control flow graph and abstract syntax tree, using self-supervised learning for feature extraction.
- **Russell et al.** [40]: a vulnerability detection method that uses Word2Vec for token embedding and applies a convolutional neural network.
- **CodeBERT**: [41]: a pre-trained model designed for source code understanding uses a bidirectional transformer to capture semantic relationships between code tokens, supporting vulnerability detection tasks.
- **VUDENC** [17]: a deep learning-based vulnerability detection tool uses a Word2Vec model and LSTM to automatically learn and identify vulnerability features in code.
- **Wang et al.** [37]: this approach combines three code representation learning models and six classification models, incorporating attention and bidirectional mechanisms for code vulnerability detection.
- **DetectVul** [42]: DetectVul applies a self-attention mechanism to directly learn patterns and interactions between statements within code functions, avoiding the need for complex graph construction.

Table 2

Python Vulnerability Datasets of Slicing Granularity: Bagheri.

granularity	Coarse-Grained			Fine-Grained			
Vulnerability type	Repo	Commits	Change file	Function	LOC	Chars	Ratio
sql inject	336	406	657	5388	83558	3960074	19.2%
xsrif	88	141	296	4418	56198	2682206	13.1%
xss	39	69	81	783	14918	736567	8.7%
command injection	85	106	197	2161	36031	1740339	12.2%
open redirect	81	93	182	1762	26521	1295748	13.3%
path disclosure	133	140	232	2968	42303	2014413	11.8%
remote code execution	50	54	131	2592	30591	1455087	9.0%

Table 3

Java Vulnerability Datasets of Line Granularity: LVDA.

Vulnerability type	count	LOC	chars
CWE-532	136,565	6,201,225	1,240,245,000
CWE-312	24,291	846,685	134,521,000
CWE-276	6,382	338,163	61,632,600
CWE-200	5,665	170,237	34,047,400
CWE-925	3,392	178,670	35,734,000
CWE-939	2,961	151,880	30,376,000
CWE-89	2036	32,691	6,138,200
CWE-79	124	6,130	126,000

Graph-based model:

- **Design** [11]: a graph neural network-based vulnerability detection method identifies potential vulnerabilities by learning the syntax and control flow graph structure of programs.
- **Reveal** [12]: Reveal uses a GGNN to extract features and completes training with a multilayer perceptron and ternary loss.
- **ReGVD** [43]: ReGVD performs vulnerability detection using a residual graph neural network, with data represented as a relationship graph between tokens created through a sliding window mechanism.
- **CSGVD** [14]: CSGVD combines sequence and graph embeddings to extract local semantic features of code through the PE-BL module, utilizes a graph neural network to extract structural information, and aggregates node information using M-BFA.

4.4. Research questions

Our evaluation was designed to answer the following five research questions:

- RQ 1: What is the effectiveness and cross-language ability of VDMAF in vulnerability detection?
- RQ 2: How does the multi-head attention feature fusion mechanism contribute to the performance of VDMAF?
- RQ 3: How do various hyperparameters in VDMAF influence the overall results?

- RQ 4: How does the globally consistent labeling generation mechanism improve the performance of VDMAF?
- RQ 5: How do different LSTM models affect the performance of VDMAF?

4.5. Evaluation metrics

We assessed the performance of VDMAF using five primary metrics: Accuracy, Precision, Recall, F1, and MCC. In this context, TP, FP, TN, and FN refer to the counts of true positives, false positives, true negatives, and false negatives, respectively. Additionally, we included three supplementary metrics: False Positive Rate (FPR), False Negative Rate (FNR), and Matthews Correlation Coefficient (MCC). The specific formulas for these metrics are outlined below.

The methods for calculating these evaluation metrics in the context of the experimental research are as follows:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (20)$$

$$Precision = \frac{TP}{TP + FP} \quad (21)$$

$$Recall = \frac{TP}{TP + FN} \quad (22)$$

$$FNR = \frac{FP}{TP + FP} \quad (23)$$

$$FNR = \frac{FP}{TP + FP} \quad (24)$$

$$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (25)$$

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (26)$$

5. Experimental result

5.1. RQ1: What is the effectiveness and cross-language ability of VDMAF in vulnerability detection?

To evaluate the effectiveness and cross-language ability of VDMAF, we evaluated the model's performance using the five metrics outlined in Section 4.5. Experiments

Table 4

Model parameters associated with each phase of VDMAF

Stage	Model	Parameter	Value	Stage	Model	Parameter	Value
I	Word2Vec	Iteration	100	III	Multi-Head Attention	Num Head	10
		Dimensionality	300			Dimension	300
		Mincount	5			Dropout	0.2
II	GGNN	Output Dim	300	IV	Classifier (Bi-LSTM)	Loss Function	WCE loss
		Learning Rate	0.01			Optimizer	Adam
		Optimizer	Adam			Learning Rate	0.01
		Epoch	100			Neurons	100
		GGNN Layer	2			Epoch	100
		Loss Function	MSE loss			Batchsize	256

were conducted on three established vulnerability datasets: Devign, Reveal, Bagheri and LVDA. For the function-level datasets Devign and Reveal, we compared VDMAF against eight widely-used vulnerability detection methods (VulDeePecker [9], SySeVR [10], Russell et al. [40], and CodeBERT [41]) and four graph-based vulnerability detection methods (Devign [11], Reveal [12], ReGVD [43], and CSGVD [14]). For the slice-level dataset Bagheri, we selected three state-of-the-art vulnerability detection methods as baselines (VUDENC [17], Wang et al. [37], DetectVul [42]). For the line-level datasets LVDA, due to the lack of excellent baseline methods in the Android source code vulnerability detection field, we used two baseline methods for C/C++ (CodeBERT and Reveal). These are the best models based on sequence embedding and graph embedding, respectively. To ensure the fairness of the comparison, we will make sure that our proposed method is consistent with all baseline training sets, test sets and validation sets.

As shown in Table 5 and Table 6, VDMAF outperformed baseline methods across both C/C++ datasets (Devign and Reveal) and the Python dataset (Bagheri), delivering higher F1 and MCC scores overall. Specifically, the F1 value of VDMAF on the Devign dataset is 65.3%, which is higher than the highest value of 55.9% of the baseline model, and on the Reveal dataset, the F1 value of VDMAF reaches 56.8%, which is higher than the performance of other models. These results demonstrate that VDMAF effectively combines sequence and graph features through the multi-head attention, enabling more accurate detection of potential vulnerabilities in code. As shown in Table 7, VDMAF outperforms CodeBERT and Reveal on the Android dataset. It shows excellent performance in accuracy (91.9%), precision (86.8%), recall (88.1%), and F1 score (87.5%), especially surpassing CodeBERT (74.0%) and Reveal (51.8%) in F1 score. These results indicate that VDMAF can more accurately identify vulnerabilities in Android platform vulnerability detection tasks and has a clear advantage in reducing false positives and false negatives.

To further summarize the advantages and limitations of VDMAF compared to existing baseline models, we compared the characteristics and performance of different models. As shown in Table 8, the table provides a detailed comparison of the models' performance in cross-language support, feature fusion, computational complexity, and vulnerability type handling, effectively showcasing VDMAF's performance and flexibility in vulnerability detection tasks.

To further evaluate VDMAF's advantages in cross-language vulnerability detection tasks, we conducted a comparison experiment based on large models. We included three popular benchmark large language models and one large model-based vulnerability detection method (GPT-3.5-turbo, GPT-4o-mini, DeepSeek-Chat, and GRACE [26]). The experimental results in Table 9 show that VDMAF outperforms these large models on all core metrics. It has clear advantages in F1 and MCC scores. This indicates that VDMAF is more accurate and stable in vulnerability detection tasks. Specifically, VDMAF achieved an F1 score of 65.3% on the Devign dataset. This is much higher than GRACE's 59.6%. The GPT series models had lower F1 scores. GPT-3.5-turbo achieved only 17.4%, and GPT-4o-mini achieved 52.7%. These results show that even though large models perform well in natural language understanding tasks, they still face challenges in source code vulnerability detection. They have limitations in extracting structured features from code, in cross-language generalization, and in the accuracy of semantic understanding.

5.2. RQ2: How does the multi-head attention feature fusion mechanism contribute to the performance of VDMAF?

In this section, we investigate the performance enhancement of the multi-head attention feature fusion mechanism in vulnerability detection tasks by comparing five distinct feature fusion methods: sum, max, mean, bi-LSTM, and our proposed VDMAF. These methods were evaluated across three classic datasets, utilizing multiple standard metrics for comparison, including Acc, Pre, Rec, F1, and MCC.

As illustrated in Table 10, the model's performance across datasets generally improves with increasing complexity of the fusion methods. VDMAF outperformed all other

Table 5

VDMAF results on the C/C++ datasets Devign and Reveal compared to each baseline method, with the best results for each metric highlighted in bold.

Metrics	Dataset					Reveal				
	Devign									
Baseline	Acc	Pre	Rec	F1	MCC	Acc	Pre	Rec	F1	MCC
VulDeePecker	55.6%	50.7%	51.2%	50.9%	0.10	84.8%	25.5%	22.9%	24.1%	0.15
SySeVR	51.5%	48.7%	52.4%	51.3%	0.10	74.2%	44.5%	21.5%	31.3%	0.12
Russell et al.	58.1%	53.5%	56.6%	55.0%	0.16	83.4%	23.9%	25.0%	24.4%	0.15
CodeBERT	63.0%	61.8%	51.0%	55.9%	0.24	92.7%	79.7%	26.6%	39.9%	0.43
Devign	58.7%	54.2%	41.2%	49.8%	0.15	89.6%	35.5%	34.8%	35.2%	0.10
Reveal	58.3%	60.4%	66.5%	61.0%	0.30	88.4%	31.1%	58.1%	39.5%	0.14
ReGVD	60.1%	57.1%	53.3%	55.1%	0.19	90.6%	47.3%	26.2%	33.7%	0.30
CSGVD	59.2%	57.3%	54.7%	56.2%	0.21	88.4%	50.2%	35.4%	39.1%	0.41
VDMAF	71.9%	62.0%	69.0%	65.3%	0.35	94.5%	40.9%	93.1%	56.8%	0.59

Table 6

Results of VDMAF on the Python dataset Bagheri versus each of the baseline methods.

F1(%)	Vulnerability Type		Dataset: Bagheri				
Baseline	SQL	XSS	CI	XSRF	RCE	PD	OR
VUDENC	80.1%	86.0%	90.5%	89.0%	88.8%	88.1%	87.3%
Wang et al.	78.9%	87.5%	88.7%	84.1%	85.9%	86.5%	83.0%
DetectVul	64.8%	42.9%	66.6%	72.3%	78.7%	63.4%	73.2%
VDMAF	96.5%	90.2%	95.1%	98.7%	98.9%	97.0%	98.8%

Table 7

Results of VDMAF on the Android(Java) dataset LVDA versus each methods.

Baseline	Acc	Pre	Rec	F1
CodeBERT	90.1%	85.2%	67.4%	74.0%
Reveal	79.6%	65.1%	85.4%	51.8%
VDMAF	91.9%	86.8%	88.1%	87.5%

methods across all three datasets. While simple linear fusion techniques such as sum and max demonstrated acceptable performance on certain datasets, they significantly lagged behind in key metrics, including precision and recall. Specifically, on the Devign dataset, VDMAF achieved an F1 score of 65.3%, representing a 7.5% improvement over the second-best mean fusion method. VDMAF's recall rate was 69.0%, slightly surpassing the bi-LSTM method's 68.8%, indicating the superior generalization capability of the multi-head attention. Regarding the MCC metric, VDMAF reached 0.35, reflecting a 9.4% increase compared to the mean fusion method's 0.32. On the Reveal dataset, VDMAF exhibited better performance, with an F1 score of 56.8%, exceeding that of other methods. In comparison to the sum fusion method's 28.2%, VDMAF's F1 score improved by 28.6%. In terms of accuracy, VDMAF achieved 94.5%, demonstrating a 2.8% enhancement over the second-best mean method and illustrating its robustness in complex datasets. On the Bagheri dataset, VDMAF excelled across all evaluation

metrics, attaining an F1 score of 97.0%, a 10.5% improvement over the bi-LSTM method. The multi-head attention effectively captures complex relationships among features through the weight allocation of multiple attention heads, thus leveraging its strengths in managing high-dimensional vulnerability detection data. Compared to traditional fusion methods, VDMAF not only outperforms baseline models in terms of performance but also demonstrates better feature fusion capabilities in multidimensional feature spaces.

5.3. RQ3: How do various hyperparameters in VDMAF influence the overall results?

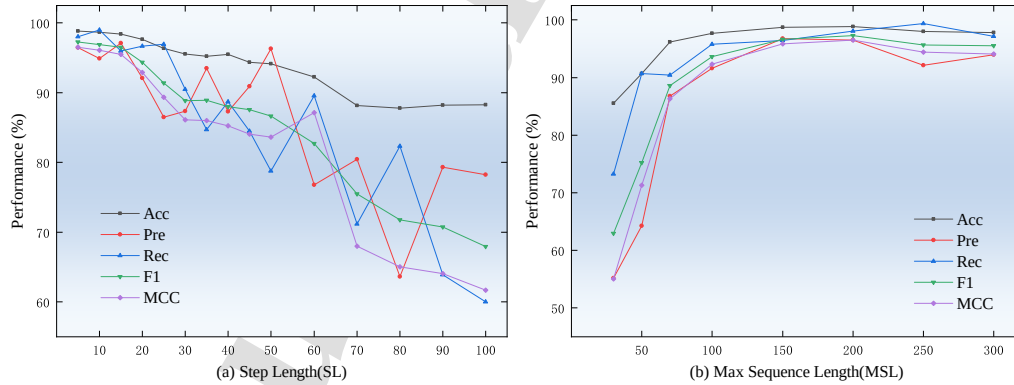
In this section, we analyze three critical hyperparameters in the VDMAF: Step Length(SL), Max Sequence Length(MSL), and Model Dimension(MD). These hyperparameters are essential for effective sample data processing and model training.

The SL parameter controls the stride of the moving focus within the source code. As shown in Fig. 7(a), increasing the SL leads to a significant decline in model performance across various metrics, including accuracy, precision, recall, F1 score, and MCC, particularly when the SL exceeds 50, where the decline becomes pronounced. This reduction in performance can be attributed to larger strides diminishing the overlap between adjacent focuses, resulting in incomplete contextual information that hinders the model's ability to effectively capture the semantic meaning of the code. To improve model efficiency, we selected a smaller SL to

Table 8

Advantages and Limitations of VDMAF vs. Existing Models.

Comparison Dimension	Cross-Language Support	Feature Fusion	Computational Complexity	Handling of Vulnerability Types
VulDeePecker	Limited (focus on C/C++)	Single feature (sequence)	Moderate	Narrow(focus on common vulnerabilities)
SySeVR	Limited (focus on C/C++)	Single feature (sequence)	Moderately high	Narrow(focus on common vulnerabilities)
Russell et al.	Limited (focus on C/C++)	Single feature (sequence)	Moderate	Narrow(focus on common vulnerabilities)
CodeBERT	Limited (focus on Python and C)	Single feature (sequence)	High	Broad (wide vulnerability range)
Devign	Limited (focus on C/C++)	Single feature (graph)	Moderately low	Narrow (focus on structural issues)
Reveal	Limited (focus on C/C++)	Single feature (graph)	Moderately high	Narrow (focus on graph-based relations)
ReGVD	Limited (focus on C/C++)	Single feature (graph)	High	Broad (handles various vulnerabilities)
CSGVD	Limited (focus on C/C++)	sequence + graph	Moderate	Broad (handles various types via graphs)
VUDENC	Limited (focus on Python)	Single feature (sequence)	Low	Moderate (focuses on specific patterns)
Wang et al.	Limited (focus on Python)	Single feature (sequence)	Moderately low	Broad (detects multiple types)
DetectVul	Limited (focus on Python)	sequence + graph	Moderate	Broad (detects multiple types)
VDMAF	Strong(C/C++,Python,Java)	sequence + graph	Moderate	Broad (SQL, XSS, CI, etc.)

**Fig. 7:** Effect of parameters Step and Max Sequence Length on results.

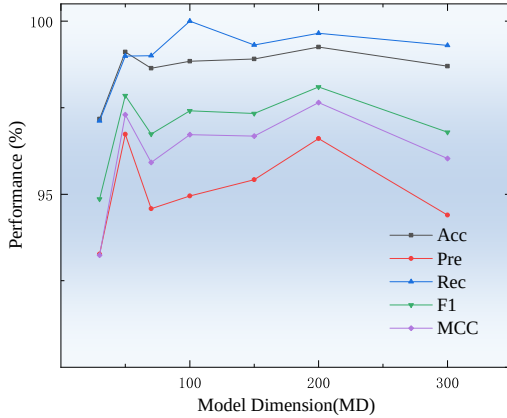
provide richer contextual information. The MSL parameter defines the size of the context window surrounding the focus in the samples. As shown in Fig. 7(b), increasing the MSL improves model performance, reaching its peak around 200. Larger context windows help to capture more features of the code, but above 200, performance plateaus or even drops slightly, mainly due to increased noise and computational

cost. Therefore, we set the MSL to 200 to achieve a balance between performance and computational overhead. The MD parameter specifies the dimensionality of the word2vec and GGNN vectors. Fig. 8 illustrates the effect of MD on model performance, revealing a gradual improvement with increasing dimensionality, peaking at 200 dimensions. Higher dimensions enhance the representation of complex code features; however, exceeding 200 dimensions may lead

Table 9

VDMAF results on the Devign compared to LLM-based method, with the best results for each metric highlighted in bold.

Baseline	Acc	Pre	Rec	F1
gpt-3.5-turbo	48.4%	54.5%	10.4%	17.4%
gpt-4o-mini	52.1%	51.8%	81.3%	52.7%
deepseek-chat	52.7%	55.8%	45.1%	49.9%
GRACE	59.2%	51.3%	71.2%	59.6%
VDMAF	71.9%	62.0%	69.0%	65.3%

**Fig. 8:** Effect of the parameter Dimension on the results.

to a risk of overfitting. Consequently, we selected 200 as the optimal dimension to ensure an effective balance between model performance and complexity.

5.4. RQ4: How does the globally consistent labeling generation mechanism improve the performance of VDMAF?

The table illustrates the model's performance in detecting various types of code vulnerabilities under different thresholds (I_v), comparing standard labeling with globally consistent threshold labeling. Specifically, we evaluated the impact of the global consistent label generation mechanism on the model's vulnerability detection, focusing on how the labeling strategy improves feature representation and enhances vulnerability context capture.

In comparison to standard labeling, the globally consistent labeling demonstrates better detection performance across all vulnerability types, as shown in Table 11. This mechanism allows the model to more accurately capture the contextual information surrounding vulnerable code, thereby enhancing the integrity of feature representation. For example, in SQL injection vulnerability detection, accuracy improved by 1.87%, the F1 score increased by 1.87%, and the MCC rose by 0.088. This mechanism particularly excels in addressing complex vulnerability types, such as remote code execution(RCE) and path disclosure(PD), suggesting that the globally consistent labeling strategy effectively captures more intricate feature relationships. In order to explore the

optimal threshold parameter I_v , we did specific ablation experiments, and the experimental results are shown in Table 12. When the threshold I_v is set to 0, the model performs relatively low on each metric, which indicates that labeling without globally consistent label generation mechanism and contextual information may lead to incomplete feature representation, thus affecting the detection performance. As the threshold increases, the model's performance based on the globally consistent labeling mechanism gradually improves. Particularly at thresholds of 50 and 100, all metrics reach high levels, indicating that moderate contextual information improves the model's vulnerability detection capabilities. When the threshold is set to 200, the model's performance improves further, achieving an MCC of 0.92. This result suggests that incorporating additional contextual information and globally consistent labels improves the model's detection accuracy and stability.

The VDMAF method proposed in this paper is based on learning vulnerability patterns. It assumes that vulnerable parts of the source code exhibit specific vulnerability patterns. By labeling vulnerable code segments, the model learns to recognize similar vulnerabilities. Even if conditional statements preventing unsafe code are not labeled, they can still serve as effective features for vulnerability mitigation. Although the labeling method may overlook some protective measures, the model can still identify potential vulnerabilities through global context modeling.

5.5. RQ5: How do different LSTM models affect the performance of VDMAF?

To evaluate the performance of different LSTM models in vulnerability detection tasks, we conducted comparative experiments on the Devign dataset, including the base LSTM and its three improved variants (BiLSTM, LSTM+attention, and BiLSTM+attention). To ensure a fair comparison, all other hyperparameters were kept consistent.

As shown in Table 13, the combination of various loss functions (mse , wce , $dwce$) with attention mechanisms had a effect on model performance across different evaluation metrics. The basic *LSTM*, using the wce loss function, showed marginally better results, with an accuracy of 66.8% and an F1 score of 60.9%. After introducing *Bi-LSTM*, the model's performance improved notably, particularly with the wce loss function, where *Bi-LSTM* achieved an accuracy of 71.9%, an F1 score of 63.6%, and an MCC of 0.41. Further analysis indicated that the addition of the attention mechanism (P_{att}) led to mixed results. In some cases, the attention mechanism substantially improved classification outcomes, such as in *Bi-LSTM* _{$wce+P_{att}$} , where recall and F1 scores increased to 71.2% and 64.0%. However, in other cases, the attention mechanism did not produce the anticipated improvements. For example, with *LSTM* _{$dwce+P_{att}$} , recall dropped to 46.6%, and the F1 score decreased to 53.9%, which could be due to the attention mechanism's failure to fully exploit both sequence and graph features.

Table 10

The results of different feature fusion methods, with the best results for each metric highlighted in bold.

Dataset	Fusion method	Acc	Pre	Rec	F1	MCC
Devign	sum	67.1%	60.0%	54.3%	57.0%	0.30
	max	67.1%	56.7%	56.7%	56.8%	0.30
	mean	67.3%	65.3%	51.8%	57.8%	0.32
	bi-lstm	65.4%	56.2%	68.8%	61.9%	0.31
	VDMAF	71.9%	62.0%	69.0%	65.3%	0.35
Reveal	sum	88.5%	19.2%	52.3%	28.2%	0.26
	max	88.9%	13.4%	43.7%	20.5%	0.19
	mean	91.7%	23.2%	58.8%	33.3%	0.33
	bi-lstm	91.5%	20.9%	56.2%	30.5%	0.30
	VDMAF	94.5%	40.9%	93.1%	56.8%	0.59
Bagheri et al.	sum	94.0%	75.9%	99.8%	86.2%	0.83
	max	92.8%	72.4%	97.4%	83.1%	0.80
	mean	93.4%	73.6%	98.6%	84.2%	0.81
	bi-lstm	94.1%	76.4%	99.8%	86.5%	0.84
	VDMAF	98.9%	96.1%	99.0%	97.0%	0.96

Table 11

Ablation study results without the threshold parameter I_v , with the best results for each metric highlighted in bold.

F1(%)	Vulnerability	Type						
Baseline	SQL	XSS	CI	XSRF	RCE	PD	OR	
w/o I_v	89.7	87.3	92.3	95.6	97.6	94.3	97.5	
VDMAF	91.6	90.2	95.1	98.7	98.9	97.0	98.8	

6. Discussion

6.1. The Impact of Sequence Features and Graph Structure Features on VDMAF Model Performance

In the VDMAF framework, combining sequence features and graph features through the multi-head attention mechanism is a core component. To evaluate the independent effects and potential redundancy of these two feature types, we conducted ablation experiments. We removed sequence features (w/o sequence) and graph features (w/o graph) to explore their unique contributions to detection performance. As shown in Table 14, VDMAF performs best when both sequence and graph features are combined. In the Devign dataset, VDMAF achieves an F1 score of 66.4%. When graph features are removed (w/o graph), the F1 score drops to 59.8%, and when sequence features are removed (w/o sequence), the F1 score drops to 63.8%. Similarly, in the Reveal dataset, VDMAF's F1 score is 56.4%. After removing graph and sequence features, the F1 score drops to 50.1% and 54.0%, respectively. Clearly, the absence of graph features has a more significant impact on model performance, while the absence of sequence features has a smaller effect. This is due to the complementary roles of sequence and graph features in capturing code information. Sequence features enhance the model's understanding of fine-grained details by capturing local semantic information, such as relationships

between variable definitions and usages. Graph features provide global context by expressing structural relationships in the code, such as function call relationships and data flow. In vulnerability detection, fine-grained semantic features are needed to identify potential unsafe statements, while global structural features help trace vulnerability propagation paths.

The multi-head attention mechanism combines the advantages of these two feature types by distributing weights across different feature subspaces. On one hand, the attention mechanism selectively emphasizes key parts of the sequence features, such as local variables and conditional statements. On the other hand, it extracts global dependencies from the graph structure and builds stronger semantic context connections. This feature fusion mechanism explains why VDMAF outperforms configurations using only a single feature type when both feature types are combined.

6.2. Exploration of computational overhead and performance balance in optimization models

This section aims to explore the possibility of optimizing computational overhead by adjusting model configurations, such as the number of attention heads and embedding dimensions, while evaluating the changes in model performance. The goal is to provide lightweight solutions for practical applications in resource-constrained scenarios. We conducted experiments on the Devign dataset using the VDMAF model, and the results are shown in Table 15. A clear trade-off between computational overhead and detection performance was observed with changes in the number of attention heads and embedding dimensions. Specifically, as the number of heads decreased from 20 to 2 and the embedding dimension reduced from 300 to 50, the model's computational time decreased significantly, but the F1 score also showed a downward trend. When the number of heads and embedding dimensions were high (e.g., 20 heads, 300 embedding dimensions), the model's feature extraction capability was stronger, achieving an F1 score of 64.8%, but

Table 12

The ablation study results exploring the optimal threshold parameter I_v , with the best results for each metric highlighted in bold. Cells with gray shading indicate the top three parameter performances for each metric, with darker shading representing better performance.

Baseline	Acc	Pre	Rec	F1	FPR	FNR	MCC
w/o I_v	97.4%	83.6%	96.9%	89.7%	24.2%	30.7%	0.88
w/ $I_v=10$	97.3%	84.5%	95.7%	89.8%	24.5%	42.4%	0.88
w/ $I_v=30$	97.3%	84.8%	96.3%	90.2%	25.3%	36.3%	0.88
w/ $I_v=50$	97.5%	86.4%	97.5%	91.6%	25.0%	24.7%	0.90
w/ $I_v=100$	97.2%	88.4%	96.6%	92.4%	26.8%	33.1%	0.90
w/ $I_v=200$	97.2%	90.4%	97.3%	93.7%	28.1%	26.9%	0.92

Table 13

The impact of different LSTM models on VDMAF in the Devign dataset, with the best results for each metric highlighted in bold. Cells with gray shading indicate the top three performing models for each metric, with darker shading representing better performance.

Classifier Model	Acc	Pre	Rec	F1	MCC
$LSTM_{mse}$	67.6%	56.2%	61.8%	58.9%	0.32
$LSTM_{wce}$	66.8%	55.4%	67.6%	60.9%	0.33
$LSTM_{dwce}$	67.1%	60.9%	56.2%	58.5%	0.31
$Bi-LSTM_{mse}$	69.5%	61.1%	60.6%	60.8%	0.35
$Bi-LSTM_{wce}$	71.9%	66.9%	60.6%	63.6%	0.41
$Bi-LSTM_{dwce}$	64.9%	59.3%	60.8%	60.1%	0.28
$LSTM_{mse} + \rho_{att}$	67.3%	60.1%	54.0%	56.9%	0.30
$LSTM_{wce} + \rho_{att}$	64.4%	63.1%	52.6%	57.4%	0.27
$LSTM_{dwce} + \rho_{att}$	68.1%	63.8%	46.6%	53.9%	0.31
$Bi-LSTM_{mse} + \rho_{att}$	67.6%	61.5%	60.0%	60.7%	0.33
$Bi-LSTM_{wce} + \rho_{att}$	68.4%	58.1%	71.2%	64.0%	0.37
$Bi-LSTM_{dwce} + \rho_{att}$	70.8%	66.9%	58.1%	62.2%	0.38

* mse refers to "Mean Squared Error Loss," wce refers to "Weighted Cross-Entropy Loss," and $dwce$ refers to "Dynamic Weighted Cross-Entropy Loss."

* ρ_{att} refers to the addition of the attention mechanism to the classification model.

the computation time was longer (269ms/t). With 8 heads and an embedding dimension of 150, the computation time dropped to 121ms/t, a 55.0% reduction compared to the initial configuration, and the F1 score improved to 66.1%, offering the best balance between computational efficiency and performance. However, when the number of heads and embedding dimensions were further reduced to 4 and below 100, the F1 score significantly dropped (e.g., F1 score of 60.7% for 4 heads and 40.5% for 2 heads), indicating that the feature extraction capability was severely affected by the

further lightweight configuration. Therefore, we recommend selecting the configuration of 8 heads and 150 embedding dimensions for lightweight setups. This configuration significantly optimizes computational overhead while maintaining high detection performance, making it suitable for practical applications with limited computational resources.

Table 14

The ablation study results for VDMAF on the C/C++ datasets, Devign and Reveal, without graph embeddings and without sequence embeddings, are shown. The best result for each metric is highlighted in bold.

Metrics	Dataset							
	Devign				Reveal			
Baseline	Acc	Pre	Rec	F1	Acc	Pre	Rec	F1
w/o graph	65.8%	56.9%	63.0%	59.8%	87.2%	34.8%	92.5%	50.1%
w/o sequence	59.1%	60.2%	68.0%	63.8%	93.4%	38.2%	91.0%	54.0%
VDMAF	72.0%	62.7%	71.2%	66.4%	94.9%	40.3%	92.8%	56.4%

Table 15

A comparative analysis of VDMAF performance and computational overhead.

Head num	Ed	F1	time	Reduced time
20	300	64.8%	269 ms/t	/
16	250	64.2%	181 ms/t	32.7%
10	200	65.3%	154 ms/t	42.8%
8	150	66.1%	121 ms/t	55.0%
4	100	60.7%	85 ms/t	68.4%
2	50	40.5%	57 ms/t	78.8%

* Ed refers to "Embedding dimensions", time refers to "computation time," and Reduced time refers to "Reduced computation time."

6.3. Threats to validity

One of the primary external threats to this study is data reliability, particularly within the field of vulnerability detection. Dataset imbalance presents a significant challenge, as demonstrated in the Reveal and Bagheri datasets, where only 8.7% to 22.9% of the code contains vulnerabilities, posing a challenge for model training. Multi-head attention may suffer from insufficient fusion when combining Word2Vec and GGNN features, which may limit the ability of the classifier to detect relevant vulnerabilities. Furthermore, if the hyperparameter tuning fails to find the optimal combination, the model performance may be degraded as a result. The study's results may also be affected by the selection of evaluation metrics. To minimize the impact of randomness, we selected three to four commonly used comprehensive evaluation metrics, ensuring the validity and reliability of the conclusions.

7. Conclusion

In current research on vulnerability detection using deep learning, sequence information helps capture fine-grained syntactic and semantic features, while structural information provides higher-level context and program flow analysis. However, there is a lack of an efficient method to combine sequence and graph structural features. Existing methods are often designed for a single language. They are less effective in vulnerability detection tasks for cross-language projects. We propose VDMAF, a vulnerability detection framework based on multi-head attention feature fusion. The framework introduces a multi-head attention module to combine sequence features and graph features. It captures semantic information between nodes and global information about code structure. The framework enables cross-language vulnerability detection by extracting unified feature representations from source code and using a multi-head attention module to focus on different feature subspaces. First, we propose an improved globally consistent labeling strategy that improves data representation by setting thresholds, enabling the model to aggregate information more effectively. Second, we use a sequence embedding strategy to capture the local semantic features of the code, and we use a graph neural network to extract its structural features. Finally, we

combine the sequence and graphical features using multi-heads attention and classify them using a BiLSTM-based recurrent neural network. To evaluate the effectiveness and generalizability of VDMAF, experiments were conducted on three vulnerability datasets with varying granularities and programming languages. The F1 scores achieved were 98.9%, 65.3%, and 56.8%, which all better than the existing baseline models. These results demonstrate that VDMAF excels in handling cross-language projects with different granularities. By introducing new methods and perspectives to code vulnerability detection, VDMAF shows potential for practical applications. Future work will systematically study the cooperative mechanisms between pre-trained models and multimodal fusion, especially decoupling the contributions of different modules when there is sufficient data support. In addition, VDMAF is expected to be applied in the security review of open-source software and automated vulnerability scanning in large-scale enterprise systems, especially in microservice architectures and containerized environments, to improve system security, reliability, and development efficiency. Future work will systematically study the cooperative mechanisms between pre-trained models and multimodal fusion, especially decoupling the contributions of different modules when there is sufficient data support.

References

- [1] Nist - cve-2023-46805 detail. <https://nvd.nist.gov/vuln/detail/CVE-2023-46805>, 2024.
- [2] Cybernews - healthcare digital platform breached, 4m+ exposed. <https://cybernews.com/news/healthcare-breach-4-million-compromised/>, 2023.
- [3] Dawson Engler, David Yu Chen, Seth Hallem, Andy Chou, and Benjamin Chelf. Bugs as deviant behavior: A general approach to inferring errors in systems code. *ACM SIGOPS Operating Systems Review*, 35(5):57–72, 2001. <https://doi.org/10.1145/502059.502041>.
- [4] Seyed Mohammad Ghaffarian and Hamid Reza Shahriari. Software vulnerability analysis and discovery using machine-learning and data-mining techniques: A survey. *ACM computing surveys (CSUR)*, 50(4):1–36, 2017. <https://doi.org/10.1145/3092566>.
- [5] Yunhui Zheng, Saurabh Pujar, Burn Lewis, Luca Buratti, Edward Epstein, Bo Yang, Jim Laredo, Alessandro Morari, and Zhong Su. D2a: A dataset built for ai-based vulnerability detection methods using differential analysis. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 111–120. IEEE, 2021. <https://ieeexplore.ieee.org/document/9402126/>.
- [6] Cybernews - finastra discloses large-scale data breach. <https://cybernews.com/news/finastra-data-breach/>, 2024.
- [7] Benjamin Steenhoek, Md Mahbubur Rahman, Richard Jiles, and Wei Le. An empirical study of deep learning models for vulnerability detection. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 2237–2248. IEEE, 2023. <https://ieeexplore.ieee.org/document/10172583>.
- [8] Arvind Kumar Bhardwaj, PK Dutta, and Pradeep Chintale. Ai-powered anomaly detection for kubernetes security: A systematic approach to identifying threats. *Babylonian Journal of Machine Learning*, 2024:142–148, 2024. <https://doi.org/10.58496/BJML/2024/014>.
- [9] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. Vuldeepecker: A deep learning-based system for vulnerability detection. *arXiv preprint arXiv:1801.01681*, 2018. <https://doi.org/10.14722/ndss.2018.23158>.
- [10] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. Sysevr: A framework for using deep learning to detect

- software vulnerabilities. *IEEE Transactions on Dependable and Secure Computing*, 19(4):2244–2258, 2021. <https://ieeexplore.ieee.org/document/9321538/>.
- [11] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. *Advances in neural information processing systems*, 32, 2019. <https://doi.org/10.48550/arXiv.1909.03496>.
- [12] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. Deep learning based vulnerability detection: Are we there yet? *IEEE Transactions on Software Engineering*, 48(9):3280–3296, 2021. <https://ieeexplore.ieee.org/document/9448435/>.
- [13] Ruitong Liu, Yanbin Wang, Haitao Xu, Jianguo Sun, Fan Zhang, Peiyue Li, and Zhenhao Guo. Vul-ImgNets: Fusing language models and online-distilled graph neural networks for code vulnerability detection. *Information Fusion*, 115:102748, 2025. <https://doi.org/10.1016/j.inffus.2024.102748>.
- [14] Wei Tang, Mingwei Tang, Minchao Ban, Zigu Zhao, and Mingjun Feng. Csgvd: A deep learning approach combining sequence and graph embedding for source code vulnerability detection. *Journal of Systems and Software*, 199:111623, 2023. <https://doi.org/10.1016/j.jss.2023.111623>.
- [15] Zhonglin Liu, Yong Fang, Cheng Huang, and Yijia Xu. Mfxss: An effective xss vulnerability detection method in javascript based on multi-feature model. *Computers & Security*, 124:103015, 2023. <https://doi.org/10.1016/j.cose.2022.103015>.
- [16] Lejun Zhang, Jinlong Wang, Weizheng Wang, Zilong Jin, Chunhui Zhao, Zhennao Cai, and Huiling Chen. A novel smart contract vulnerability detection method based on information graph and ensemble learning. *Sensors*, 22(9):3581, 2022. <https://doi.org/10.3390/s22093581>.
- [17] Laura Wartschinski, Yannic Noller, Thomas Vogel, Timo Kehler, and Lars Grunske. Vudenc: vulnerability detection with deep learning on a natural codebase for python. *Information and Software Technology*, 144:106809, 2022. <https://doi.org/10.1016/j.infsof.2021.106809>.
- [18] Lida Zhao, Sen Chen, Zhengzi Xu, Chengwei Liu, Lyuye Zhang, Jiahui Wu, Jun Sun, and Yang Liu. Software composition analysis for vulnerability detection: An empirical study on java projects. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 960–972, 2023. <https://doi.org/10.1145/3611643.3616299>.
- [19] Huanting Wang, Guixin Ye, Zhanyong Tang, Shin Hwei Tan, Songfang Huang, Dingyi Fang, Yansong Feng, Lizhong Bian, and Zheng Wang. Combining graph-based learning with automated data collection for code vulnerability detection. *IEEE Transactions on Information Forensics and Security*, 16:1943–1958, 2020. <https://ieeexplore.ieee.org/document/9293321/>.
- [20] Tianwei Lei, Jingfeng Xue, Yong Wang, and Zhenyan Liu. Irc-clvl: Cross-programming-language vulnerability detection with intermediate representations and combined features. *Electronics*, 12(14):3067, 2023. <https://doi.org/10.3390/electronics12143067>.
- [21] Deqing Zou, Sujuan Wang, Shouhuai Xu, Zhen Li, and Hai Jin. uvuldeepecker: A deep learning-based system for multiclass vulnerability detection. *IEEE Transactions on Dependable and Secure Computing*, 18(5):2224–2236, 2019. <https://ieeexplore.ieee.org/document/8846081/>.
- [22] Sicong Cao, Xiaobing Sun, Lili Bo, Ying Wei, and Bin Li. Bgmn4vd: Constructing bidirectional graph neural-network for vulnerability detection. *Information and Software Technology*, 136:106576, 2021. <https://doi.org/10.1016/j.infsof.2021.106576>.
- [23] Sahil Suneja, Yunhui Zheng, Yufan Zhuang, Jim Laredo, and Alessandro Morari. Learning to map source code to software vulnerability using code-as-a-graph. *arXiv preprint arXiv:2006.08614*, 2020. <https://doi.org/10.48550/arXiv.2006.08614>.
- [24] Min Li, Chunfang Li, Shualou Li, Yanna Wu, Boyang Zhang, and Yu Wen. Acgvd: Vulnerability detection based on comprehensive graph via graph neural network with attention. In *Information and Communications Security: 23rd International Conference, ICICS 2021, Chongqing, China, November 19-21, 2021, Proceedings, Part I 23*, pages 243–259. Springer, 2021. https://link.springer.com/chapter/10.1007/978-3-030-86890-1_14.
- [25] Guixin Ye, Zhanyong Tang, Huanting Wang, Dingyi Fang, Jianbin Fang, Songfang Huang, and Zheng Wang. Deep program structure modeling through multi-relational graph-based learning. In *Proceedings of the ACM International conference on parallel architectures and compilation techniques*, pages 111–123, 2020. <https://doi.org/10.1145/3410463.3414670>.
- [26] Guilong Lu, Xiaolin Ju, Xiang Chen, Wenlong Pei, and Zhilong Cai. Grace: Empowering llm-based software vulnerability detection with graph structure and in-context learning. *Journal of Systems and Software*, 212:112031, 2024. <https://doi.org/10.1016/j.jss.2024.112031>.
- [27] Yi Li, Shao-hua Wang, and Tien N Nguyen. Vulnerability detection with fine-grained interpretations. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 292–303, 2021. <https://doi.org/10.1145/3468264.3468597>.
- [28] Zhenhao Luo, Pengfei Wang, Baosheng Wang, Yong Tang, Wei Xie, Xu Zhou, Danjun Liu, and Kai Lu. Vulhawk: Cross-architecture vulnerability detection with entropy-based binary code search. In *NDSS*, 2023. https://www.ndss-symposium.org/wp-content/uploads/2023/02/ndss2023_f415_paper.pdf.
- [29] Zhihui Song, Jinchun Xu, Kewei Li, and Zheng Shan. Hcrvd: A vulnerability detection system based on cst-pdg hierarchical code representation learning. *Computers, Materials & Continua*, 79(3), 2024. <https://www.techscience.com/cmc/v79n3/57105>.
- [30] Son Nguyen, Thu-Trang Nguyen, Thanh Trong Vu, Thanh-Dat Do, Kien-Tuan Ngo, and Hieu Dinh Vo. Code-centric learning-based just-in-time vulnerability detection. *Journal of Systems and Software*, 214:112014, 2024. <https://doi.org/10.1016/j.jss.2024.112014>.
- [31] Zhilong Cai, Yongwei Cai, Xiang Chen, Guilong Lu, Wenlong Pei, and Junjie Zhao. Cvsd-tf: Cross-project software vulnerability detection with tradaboost by fusing expert metrics and semantic metrics. *Journal of Systems and Software*, 213:112038, 2024. <https://doi.org/10.1016/j.jss.2024.112038>.
- [32] Xin-Cheng Wen, Yupan Chen, Cuiyun Gao, Hongyu Zhang, Jie M Zhang, and Qing Liao. Vulnerability detection with graph simplification and enhanced graph representation learning. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 2275–2286. IEEE, 2023. <https://ieeexplore.ieee.org/document/10172762>.
- [33] Wangshu Liu, Ye Yue, Xiang Chen, Qing Gu, Pengzhan Zhao, Xuejun Liu, and Jianjun Zhao. Sedpgk: Semi-supervised software defect prediction with graph representation learning and knowledge distillation. *Information and Software Technology*, page 107510, 2024. <https://doi.org/10.1016/j.infsof.2024.107510>.
- [34] Wei Xiao, Zhengzhang Hou, Tao Wang, Chengxian Zhou, and Chao Pan. Msgvul: Multi-semantic integration vulnerability detection based on relational graph convolutional neural networks. *Information and Software Technology*, page 107442, 2024. <https://doi.org/10.1016/j.infsof.2024.107442>.
- [35] Chunyong Zhang, Tianxiang Yu, Bin Liu, and Yang Xin. Vulnerability detection based on federated learning. *Information and Software Technology*, 167:107371, 2024. <https://doi.org/10.1016/j.infsof.2023.107371>.
- [36] Kui Liu, Dongsun Kim, Tegawendé F Bissyandé, Shin Yoo, and Yves Le Traon. Mining fix patterns for findbugs violations. *IEEE Transactions on Software Engineering*, 47(1):165–188, 2018. <https://ieeexplore.ieee.org/abstract/document/8565907>.
- [37] Rongcun Wang, Senlei Xu, Xingyu Ji, Yuan Tian, Lina Gong, and Ke Wang. An extensive study of the effects of different deep learning models on code vulnerability detection in python code. *Automated Software Engineering*, 31(1):15, 2024. <https://link.springer.com/article/10.1007/s10515-024-00413-4>.
- [38] Joern - the bug hunter's workbench. <https://joern.io/>, 2023.

- [39] Janaka Senanayake, Harsha K Kalutarage, M Omar Al-Kadri, Luca Piras, and Andrei Petrovski. Labelled vulnerability dataset on android source code (Ivdandro) to develop ai-based code vulnerability detection models. In *SECRYPT*, pages 659–666, 2023. <https://www.scitepress.org/Papers/2023/120604/120604.pdf>.
- [40] Rebecca Russell, Louis Kim, Lei Hamilton, Tomo Lazovich, Jacob Harer, Onur Ozdemir, Paul Ellingwood, and Marc McConley. Automated vulnerability detection in source code using deep representation learning. In *2018 17th IEEE international conference on machine learning and applications (ICMLA)*, pages 757–762. IEEE, 2018. <https://ieeexplore.ieee.org/document/8614145/>.
- [41] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155*, 2020. <https://doi.org/10.48550/arXiv.2002.08155>.
- [42] Hoai-Chau Tran, Anh-Duy Tran, and Kim-Hung Le. Detectvul: A statement-level code vulnerability detection for python. *Future Generation Computer Systems*, page 107504., 2024. <https://doi.org/10.1016/j.future.2024.107504>.
- [43] Van-Anh Nguyen, Dai Quoc Nguyen, Van Nguyen, Trung Le, Quan Hung Tran, and Dinh Phung. Regvd: Revisiting graph neural networks for vulnerability detection. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, pages 178–182., 2022. <https://doi.org/10.1145/3510454.3516865>.

Declaration of interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.